



Refatoração do Bioinspired Knapsack

ALGORITMOS AVANÇADOS

ALUNOS:

GUILHERME ELIAS KOPSCH

GUILHERME MACHADO

RUAN CARDOZO

Introdução

main

1 Branch

0 Tags

Go to file

+

<> Code

v0cs

Update README.md

413c2bd · 2 weeks ago

3 Commits

src	feat: adiciona estrutura inicial da solução com algoritmo g...	3 weeks ago
.gitignore	feat: adiciona estrutura inicial da solução com algoritmo g...	3 weeks ago
README.md	Update README.md	2 weeks ago
main.py	feat: adiciona estrutura inicial da solução com algoritmo g...	3 weeks ago

README

✎

☰

Problema da Mochila com Algoritmos Bio-Inspirados

Solução para o clássico problema da Mochila 0/1 (Knapsack) utilizando Python e algoritmos bio-inspirado

🔗 [Link da documentação](#)

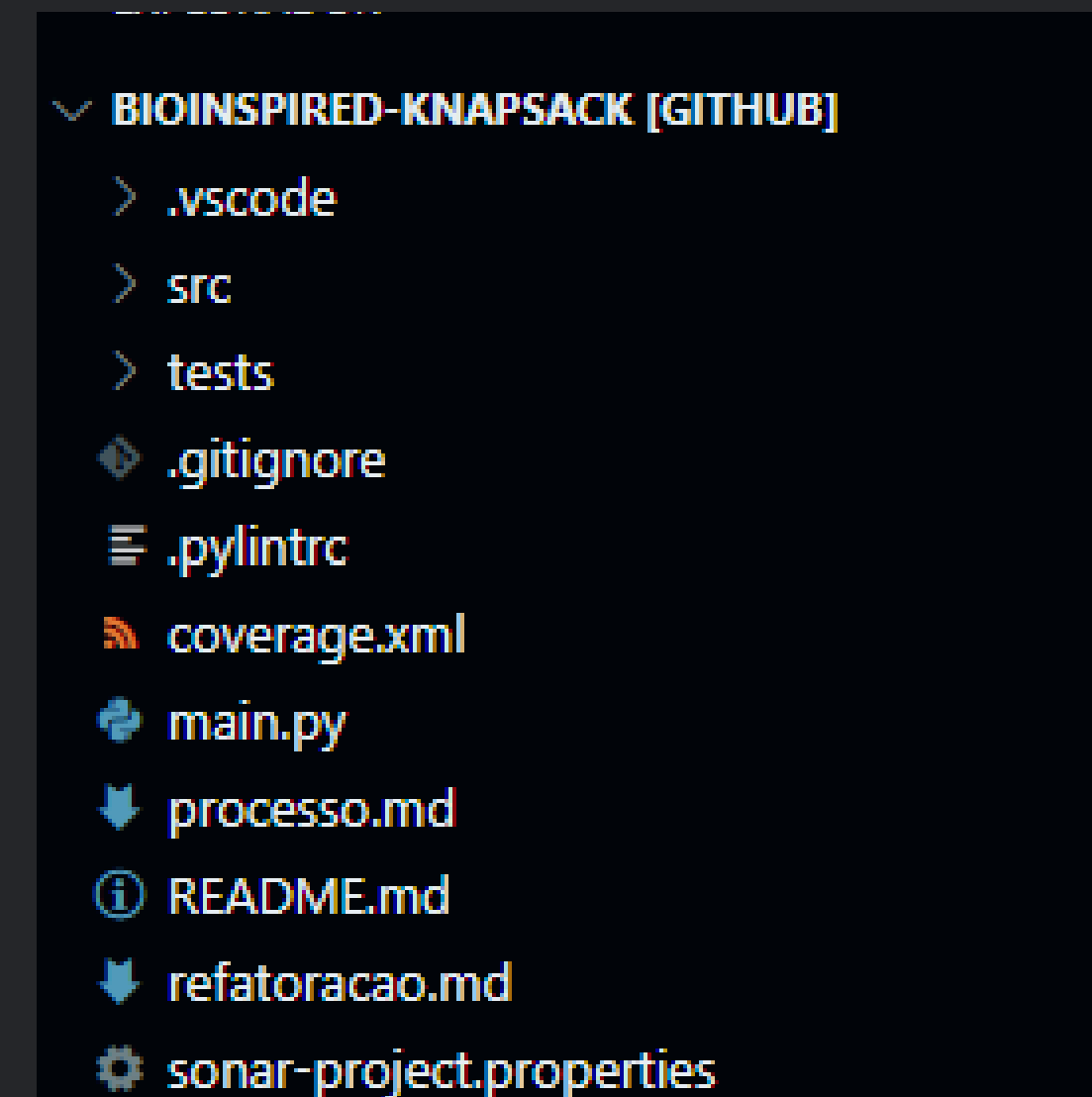
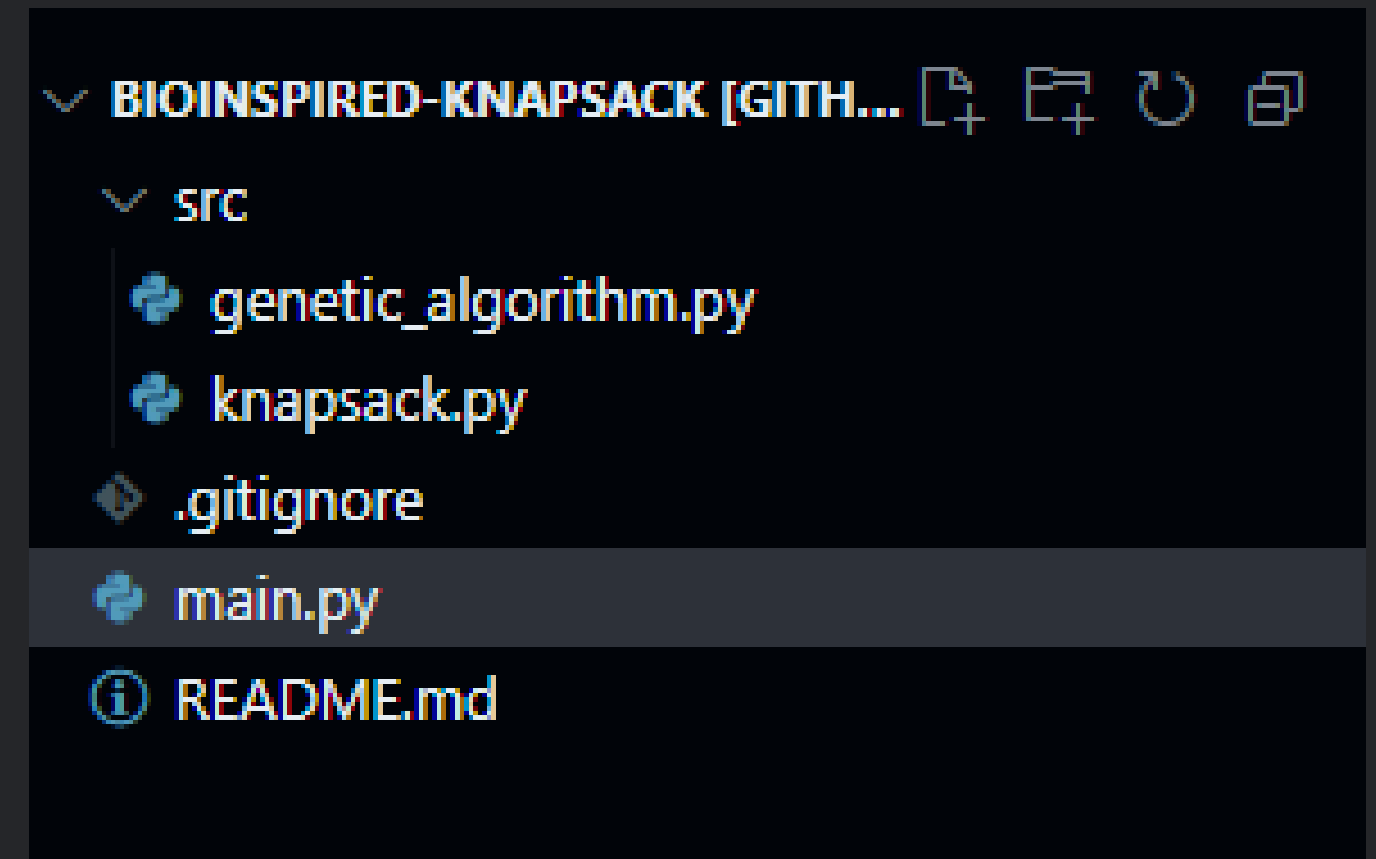


Autores

- Humberto Peres da Rocha Filho
- Leandro Pellegrini Fodi
- Vítor de Oliveira Celestino da Silva
- Weslly Hendler Neres

O que foi refatorado ?

- Funções utilitárias de entrada → extraídas para utils.
- Separação da lógica de entrada/saída → criada io.py.
- Testes unitários adicionados → tests/ para knapsack, genetic_algorithm, io, utils.



Por que foi refatorado ?

- Reduzir duplicação: centralização de funções.
- Melhorar organização: separação clara entre lógica de negócio e entrada/saída.
- Aumentar coesão e confiabilidade: cada módulo com responsabilidade única.
- Facilitar manutenção: menor acoplamento, mais testes.

Antes da Refatoração

```
bioinspired-knapsack/  
├── main.py  
├── knapsack.py  
├── genetic_algorithm.py  
└── (pouca ou nenhuma separação de módulos)
```

- Toda a lógica de entrada/saída, validação, utilidades e execução principal estava concentrada em poucos arquivos.
- Não havia separação clara entre lógica de negócio, utilidades e interface.
- Testes unitários ausentes ou misturados.

Depois da Refatoração

```
bioinspired-knapsack/  
├── main.py                # Ponto de entrada, apenas orquestração  
├── src/  
│   ├── config.py         # Instância única do gerador aleatório  
│   ├── knapsack.py        # Lógica do problema da mochila  
│   ├── genetic_algorithm.py # Algoritmo genético e operadores  
│   ├── io/  
│   │   ├── io.py         # Funções de entrada/saída e validação  
│   │   └── utils/  
│   │       └── utils.py   # Funções utilitárias (ex: geração de itens)  
├── tests/  
│   ├── knapsack_test.py  
│   ├── genetic_algorithm_test.py  
│   ├── io_test.py  
│   └── utils_test.py  
├── refatoracao.md        # Documentação da refatoração  
└── processo.md           # Processo de refatoração adotado
```

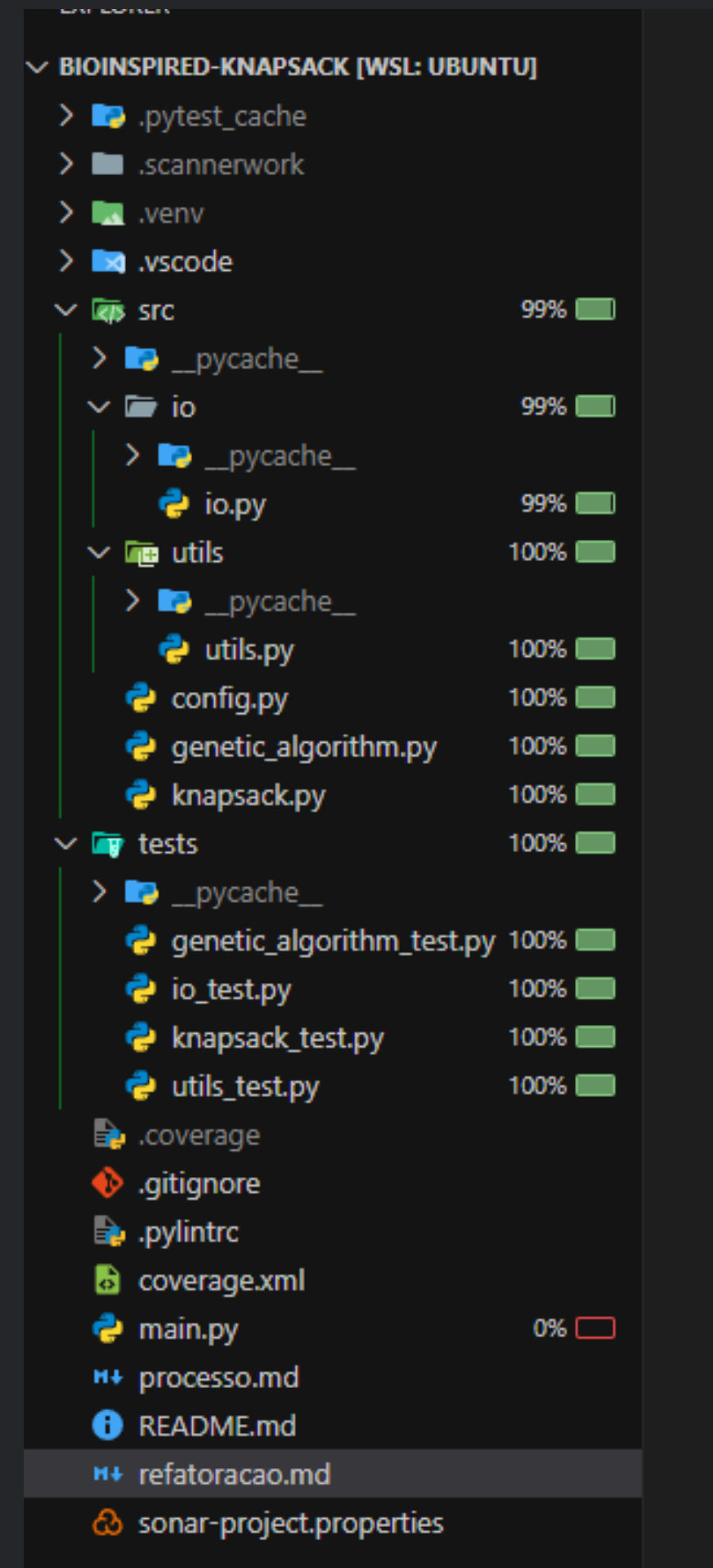
- `main.py` ficou enxuto, delegando responsabilidades para módulos especializados.
- `config.py` centraliza a configuração do gerador aleatório.
- `io.py` concentra toda a lógica de entrada/saída e validação.
- `utils.py` armazena funções utilitárias reutilizáveis.
- `knapsack.py` e `genetic_algorithm.py` focam apenas na lógica principal.
- `tests/` contém todos os testes unitários organizados por módulo.

Técnicas aplicadas

Técnica	Descrição breve	Arquivo(s) afetado(s)
Extract Method	Extrair código repetido em funções próprias	config.py(numpy random generator)
Move Function	Mover funções de utilidade para um novo módulo	main.py → io.py / utils.py
Replace Magic Number	Substituir números mágicos por constantes nomeadas	src/genetic_algorithm.py, src/knapsack.py, src/io.py
Write Unit Tests	Criar testes unitários para funções principais	tests/knapsack_test.py, tests/genetic_algorithm_test.py tests/io_test.py tests/util_test.py

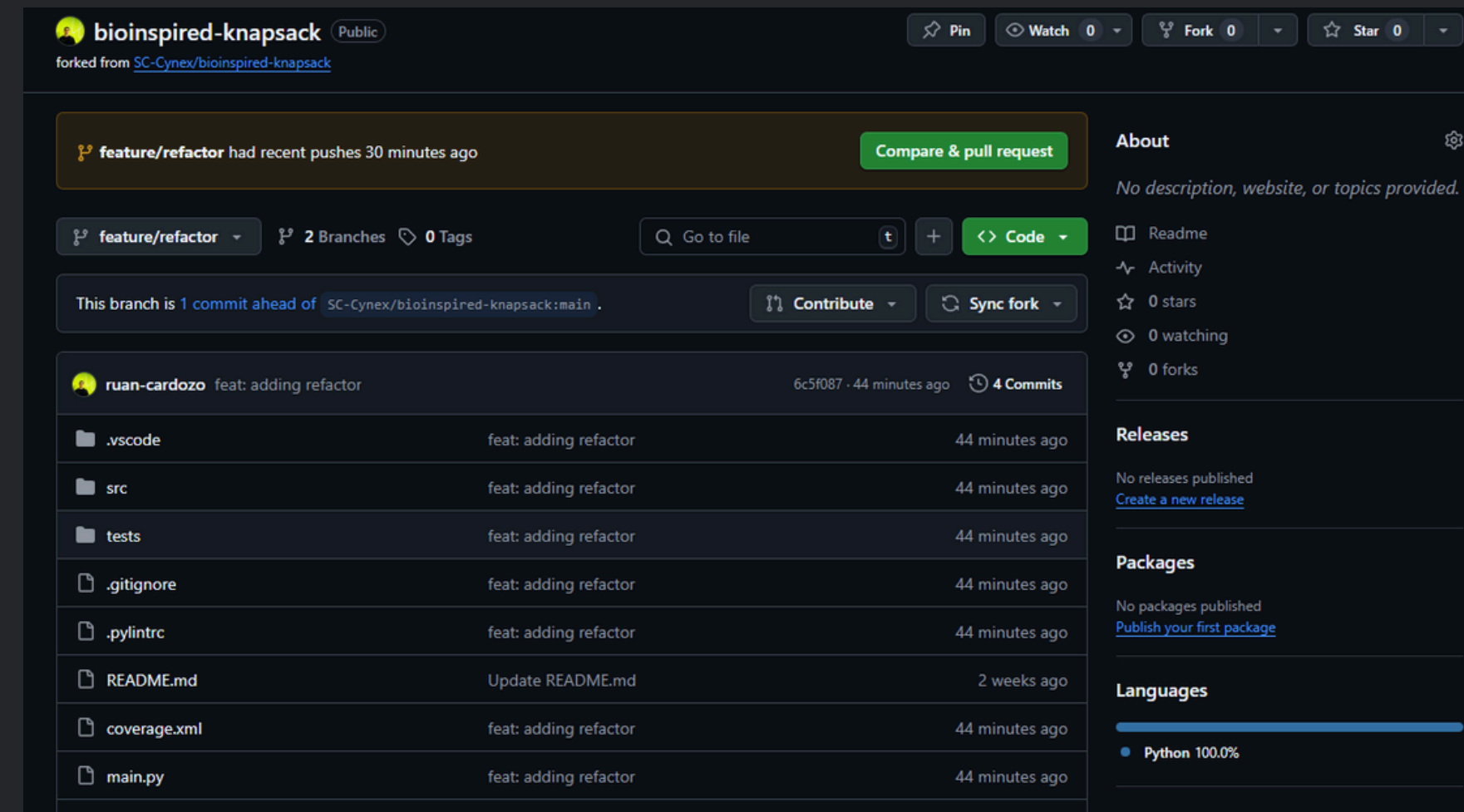
Ferramentas utilizadas

- Python padrão
- Organização modular de código
- SonarQube para análise de qualidade
- Pytest/Unittesting para testes unitários
- Pylint para linting e verificação de estilo de código
- Github Copilot para sugestões de refatoração e melhorias



Processo do Trabalho

- **Fork do repositório:** [Link](#)
- **Branch específica:** feature/refactor
- **Justificativa:** Segurança, colaboração e organização.
- **Garantia:** Código original preservado via branch e fork.



Antes da refatoração

bioinspired-knapsack/

|——src/

|——knapsack.py

|——genetic_algorithm.py

|——main.py

- Toda a lógica de entrada/saída, validação, utilidades e execução principal estava concentrada em poucos arquivos.
- Não havia separação clara entre lógica de negócio, utilidades e input/output.
- Testes unitários ausentes.

Depois da refatoração

- **main.py** ficou enxuto, delegando responsabilidades para módulos especializados.
- **config.py** centraliza a configuração do gerador aleatório(numpy).
- **io.py** concentra toda a lógica de entrada/saída e validação.
- **utils.py** armazena funções utilitárias reutilizáveis.
- **knapsack.py** e **genetic_algorithm.py** focam apenas na lógica principal.
- **tests/** contém todos os testes unitários organizados por módulo.

bioinspired-knapsack/

```
├── main.py
├── src/
│   ├── config.py
│   ├── knapsack.py
│   ├── genetic_algorithm.py
│   ├── io/
│   │   └── io.py
│   └── utils/
│       └── utils.py
├── tests/
│   ├── knapsack_test.py
│   ├── genetic_algorithm_test.py
│   ├── io_test.py
│   └── utils_test.py
├── refatoracao.md
└── processo.md
```

Ponto de entrada, apenas orquestração

Instância única do gerador aleatório

Lógica do problema da mochila

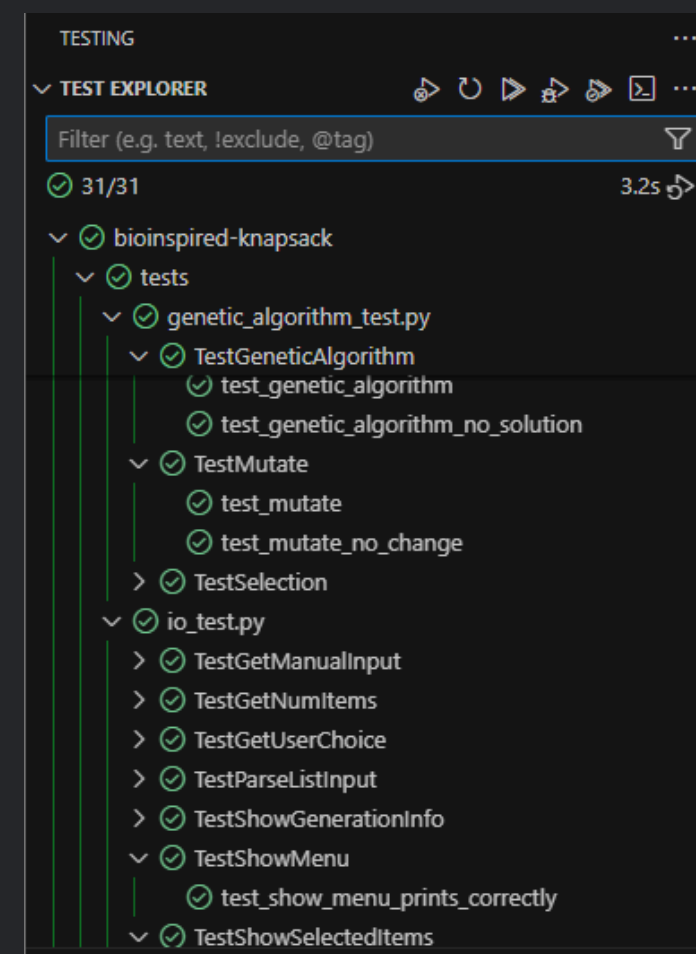
Algoritmo genético e operadores

Funções de entrada/saída e validação

Funções utilitárias (ex: geração de itens)

Documentação da refatoração

Processo de refatoração adotado



TestShowSelectedItems	
TEST COVERAGE	
bioinspired-knapsack	95.12%
main.py	0.00%
src	99.47%
config.py	100.00%
genetic_algorithm.py	100.00%
io	99.03%
io.py	99.03%
knapsack.py	100.00%
utils	100.00%
utils.py	100.00%
tests	100.00%
genetic_algorithm_test.py	100.00%
io_test.py	100.00%
knapsack_test.py	100.00%
utils_test.py	100.00%



Obrigado